

บทที่ 7

ไคเร็กอินพุต (DirectInput)

7.1 แนะนำไคเร็กอินพุต

ไคเร็กอินพุต คือ แอปพลิเคชันโปรแกรมมิ่งอินเทอร์เฟซ (API) สำหรับการใช้งานอุปกรณ์รับข้อมูล เช่น เมาส์, คีย์บอร์ด, จอยสติค และอุปกรณ์ควบคุมเกมอื่นๆ และอุปกรณ์ที่รับและส่งข้อมูลได้ เช่น อุปกรณ์ประเภทแรงส่งกลับ (force-feedback device)

7.1.1 ประสิทธิภาพของไคเร็กอินพุต (The Power of DirectInput)

นอกจากจะสนับสนุนการใช้งานบางอย่างที่ไม่สามารถใช้งานโดยผ่าน Win32 API ได้แล้ว ไคเร็กอินพุตยังเข้าถึงข้อมูลได้เร็วกว่า เนื่องจากการติดต่อโดยตรงกับฮาร์ดแวร์ไดรเวอร์ซึ่งไม่ต้องเสียเวลากับการทำให้เป็นเมสเสจด้วยระบบปฏิบัติการวินโดวส์

ไคเร็กอินพุตให้โปรแกรมสามารถรับข้อมูลจากอุปกรณ์รับข้อมูลแม้ว่ากำลังทำงานอยู่ในแบ็กกราวนด์ นอกจากนั้น ยังสนับสนุนการใช้งานอุปกรณ์ทุกชนิดรวมทั้งอุปกรณ์ประเภทฟอร์สฟีดแบ็กด้วย

โปรแกรมสามารถรับข้อมูลจากอุปกรณ์รับข้อมูลได้โดยไม่มีความจำเป็นจะต้องรู้ว่าการกำลังติดต่อกับอุปกรณ์ชนิดไหนที่สร้างข้อมูลนั้นขึ้นมา โดยใช้วิธีการแมปการกระทำ (action mapping)

ความสามารถและประสิทธิภาพที่มีมากกว่า Win32API ทำให้ไคเร็กอินพุตมีความสำคัญอย่างยิ่งในการพัฒนาโปรแกรมประเภทเกม, การจำลองสถานการณ์ และโปรแกรมที่อินเทอร์แอคทีฟกับผู้ใช้ในรูปแบบเรียลไทม์

ถ้าแอปพลิเคชันใช้คีย์บอร์ดในการรับตัวอักษรและเมาส์ในการเคลื่อนที่ไปยังบริเวณต่างๆ การใช้ไคเร็กอินพุตไม่มีประโยชน์ใดๆ เพิ่มขึ้น

7.1.2 เริ่มต้นกับไคเร็กอินพุต (Getting Started with DirectInput)

ในการที่จะเข้าใจไคเร็กอินพุตจะต้องเข้าใจในสิ่งเหล่านี้ก่อน

- วัตถุไคเร็กอินพุต (DirectInput object)
- อุปกรณ์ (Device : keyboard, mouse, joystick)
- วัตถุ DirectInputDevice
- วัตถุ Device

ต่อไปนี้เป็นขั้นตอนในการใช้งานไคเร็กอินพุต

1. สร้างวัตถุไคเร็กอินพุต

เนื่องจากจะต้องใช้เมทอดของออปเจกต์นี้ในการตรวจสอบและสร้าง DirectInput device objects

2. ตรวจสอบ Device

ถ้าต้องการจะใช้งานเพียงแค่มouseและคีย์บอร์ดซึ่งมีอยู่ในเครื่องคอมพิวเตอร์ทุกเครื่องก็ไม่จำเป็นต้องตรวจสอบ แต่ในกรณีที่ต้องการใช้อุปกรณ์อื่นๆ ที่อาจจะอยู่ในเครื่องคอมพิวเตอร์ก็ต้องทำการตรวจสอบก่อน ถ้า DirectInput พบอุปกรณ์ที่ตรงกับที่เรากำหนดมันจะทำให้เราสามารถตรวจสอบคุณสมบัติของอุปกรณ์ก่อนได้ และจะให้ค่าเฉพาะสำหรับอุปกรณ์นั้นไว้เพื่อใช้ในการสร้าง DirectInput device object เพียงแทนอุปกรณ์นั้นๆ

3 สร้าง DirectInputDevice object สำหรับแต่ละอุปกรณ์ที่ต้องการ

ที่ค่าเฉพาะสำหรับอุปกรณ์ที่ได้รับมาระหว่างการตรวจสอบจะถูกใช้ในการสร้าง แต่ถ้าใช้คีย์บอร์ดและเมาส์สามารถใช้ค่า GUID มาตรฐานแทนได้

4. เซตอัพอุปกรณ์ (Set up the device)

สำหรับแต่ละอุปกรณ์ ในขั้นตอนแรกจะต้องกำหนดระดับในการใช้งานอุปกรณ์ (cooperative level) ซึ่งเป็นตัวกำหนดวิธีการที่อุปกรณ์จะถูกใช้งานร่วมกับแอปพลิเคชันอื่นๆ อย่างไร นอกจากนั้นจะต้องกำหนดรูปแบบของข้อมูลที่จะใช้ในการจำแนก device object เช่น button, axe ใน data packet หรือถ้าต้องการในรูปของ buffer data ก็ต้องกำหนดขนาดของบัฟเฟอร์ และยังสามารถหาข้อมูลเกี่ยวกับอุปกรณ์และกำหนดคุณสมบัติต่างๆ เช่น ระยะของค่าที่ส่งมาจากจอยสติค เป็นต้น

5. การได้รับอุปกรณ์ (Acquiring device) ในขั้นตอนนี้เป็นการบอก DirectInput พร้อมที่จะรับข้อมูลจากอุปกรณ์แล้ว

6. การรับข้อมูล (Retrieve data)

เป็นการรับค่าสถานะของอุปกรณ์ในขณะนั้นหรือรับบันทึกเหตุการณ์ที่เกิดขึ้นในช่วงเวลาหลังจากการรับค่าครั้งล่าสุด หรือ จะให้ DirectInput เตือนขึ้นมาเองก็ได้

7. การประมวลผลข้อมูล (Act on the data)

แอปพลิเคชันนำข้อมูลไปประมวลผล

8. ปิดไคลเร็กอินพุต (Close DirectInput)

แต่นี้ก็ไม่ใช่วิธีการเดียวในการใช้งาน DirectInput เรายังสามารถใช้ action mapping ได้อีกเช่นกัน ด้วยการกำหนด action map ให้กับอุปกรณ์ ก็สามารถนำ device object ไปใช้สำหรับแต่ละค่าในแอปพลิเคชันได้

7.2 ทำความเข้าใจไคลเร็กอินพุต (Understanding DirectInput)

DirectInput Object

เฉพาะอุปกรณ์รับข้อมูลที่ได้รับข้อมูลได้เพียงอย่างเดียวใน DirectInput สร้างขึ้นโดยประกอบด้วย DirectInput object ซึ่งอยู่ใน IDirectInput8 COM Interface และ DirectInputDevice object สำหรับแต่ละอุปกรณ์ที่จะให้ข้อมูล ซึ่งทุกตัวจะต้องมี device object ซึ่งแทน key, button เป็นต้น (device object instance)

DirectInputDevice object อยู่ใน IDirectInputDevice8 interface ซึ่งจำนวนและชนิดของอุปกรณ์สามารถตรวจสอบได้โดยเรียกเมทอด IDirectInputDevice8::EnumObjects ค่าที่อธิบายอุปกรณ์แต่ละตัวจะเก็บอยู่ใน DIDEVICEOBJECTINSTANCE

Force-feedback effect อยู่ใน IDirectInputEffect interface ซึ่งมีเมทอดที่ใช้ในการสร้าง, เปลี่ยนแปลง, เริ่ม และหยุด การทำเอฟเฟ็ค

7.3 การติดต่อกับวินโดวส์ (Interaction with Windows)

เนื่องจาก DirectInput เป็นการทำงานโดยตรงกับ device driver ดังนั้นในบางครั้งมันอาจจะไม่สนใจ mouse และ keyboard message และ ไม่สนใจค่ากำหนดของ mouse และ keyboard ที่กำหนดไว้ใน control panel ด้วย

DirectInput ไม่มีการเห็นค่ากำหนดในการเข้าตัวอักษรสำหรับคีย์บอร์ด เมื่อใช้ buffer data DirectInput จะแปลสัญญาณเป็นการกดหรือการปล่อย 1 เหตุการณ์ไม่มีการซ้ำ เมื่อใช้ immediate data DirectInput จะสนใจเฉพาะค่าสถานะปัจจุบันของคีย์เท่านั้น นั่นคือ เหตุการณ์ของคีย์บอร์ดจะไม่มีการประมวลผลโดยผ่านวินโดวส์

DirectInput ไม่ได้ทำการแปลงหรือเปลี่ยนค่าของตัวอักษรใดๆ ทั้งสิ้น เช่น ปุ่ม SHIFT จะได้รับการปฏิบัติเช่นเดียวกับปุ่มอื่นๆ ไม่ใช้การเปลี่ยนค่าของข้อมูลที่ได้รับจากปุ่มอื่นๆ นอกจากนั้น ค่าที่ส่งกลับมามีเหมือนกันไม่ว่าจะมีการกำหนดค่าภาษาเป็นเช่นไร

ใน Windows 2000 การรับค่าของคีย์บอร์ดในแบบ exclusive จะป้องกัน แอปพลิเคชัน อื่นที่จะทำงานขึ้นมาภายหลังเมื่อได้รับข้อมูลจากคีย์บอร์ดได้

เนื่องจาก directinput ทำงานโดยตรงกับ mouse driver นั่นคือ ข้ามระบบย่อยของวินโดวส์ที่ทำหน้าที่ในการแปลงข้อมูลของเมาส์ให้กับแอปพลิเคชัน ดังนั้น ถ้าแอปพลิเคชันยังคงต้องการใช้ตัวชี้ของวินโดวส์ในการใช้งานควรใช้ standard windows mouse messages และ Microsoft ® Win32® function

เมื่อใช้ระบบเมาส์ในแบบ exclusive mode DirectInput จะเข้าไปบังระบบ mouse message และทำให้วินโดวส์ไม่สามารถที่จะแสดงตัวชี้มาตรฐานได้

DirectInput ไม่สนใจค่าที่กำหนดใน control panel เช่น ค่าความเร่ง และการแปลงค่าปุ่ม อย่างไรก็ตาม DirectInput จะยังคงเห็นค่าที่กำหนดไว้ใน Driver เอง เช่น ถ้ามีเมาส์แบบ 3 ปุ่ม และ มีการใช้งาน driver utility software เพื่อทำให้ปุ่มกลางกลายเป็น double click DirectInput จะบอกว่ามีการกดปุ่มกลางเป็นการคลิกปุ่มเมาส์ 2 ครั้ง

7.4 การใช้งาน DirectInput

1. CreateDirectInput

ขั้นตอนแรกในการใช้งาน DirectInput คือการรับค่าของ IDirectInput 8 interface ซึ่งสามารถทำได้โดยเรียกฟังก์ชัน DirectInput8Create

ในการใช้งานควรสร้าง DirectInput object เพียงตัวเดียวและปล่อยค่าเมื่อจบการทำงาน

2. DirectInput Device Enumeration

DirectInput สามารถถามระบบถึงอุปกรณ์อินพุตที่มีอยู่ทั้งหมดได้ และสามารถตรวจสอบได้ว่า อุปกรณ์เชื่อมต่ออยู่หรือไม่ และส่งค่าข้อมูลของอุปกรณ์เหล่านั้นกลับคืนมา กระบวนการนี้เรียกว่า การ ระบุ (enumeration)

ถ้าแอปพลิเคชันต้องการใช้งานเฉพาะอุปกรณ์มาตรฐาน เช่น เมาส์ หรือ คีย์บอร์ด ก็ไม่จำเป็นที่จะต้องทำการระบุอุปกรณ์ เนื่องจากสามารถใช้ค่าตัวแปรที่กำหนดไว้แล้ว (predefined global variable) เมื่อ เรียกเมทอด `Idirectinput8::CreateDevice`

อุปกรณ์ชนิดอื่นๆ และระบบที่มีคีย์บอร์ดหรือเมาส์หลายตัว จะต้องเรียก

`IdirectInput8::EnumDevices` หรือ `IdirectInput8::EnumDevicesBySemantics` ในการระบุอุปกรณ์ที่มีอยู่ และหาอุปกรณ์ตัวใดตัวหนึ่งที่เหมาะสมที่สุด หรือจะใช้ข้อมูลที่ได้มาเป็นตัวเลือกก็ได้ ตัวอย่าง code

```
GUID KeyboardGUID = GUID_SysKeyboard;
```

```
lpdi->EnumDevices(DIDEVTYPE_KEYBOARD, DEnumDevicesCallback,  
    &KeyboardGUID, DIEDFL_ATTACHEDONLY);
```

พารามิเตอร์ตัวแรก	คือ ชนิดของอุปกรณ์ที่จะทำการระบุ สามารถใส่ค่าเป็น NULL ได้ถ้าต้องการที่จะระบุอุปกรณ์ทุกชนิด หรือจะกำหนดค่าเจาะจงโดยชนิดของอุปกรณ์ก็ได้ โดยใช้ค่า <code>DIDEVTYPE_*</code> ตัวใดตัวหนึ่งซึ่งค่าที่เป็นองค์ประกอบได้กำหนดไว้แล้วใน <code>DIDeviceInstance</code>
พารามิเตอร์ตัวที่สอง	คือ ค่าพอยเตอร์ที่ชี้ไปยัง callback function ที่จะถูกเรียกเมื่อมีการระบุอุปกรณ์แต่ละตัว ฟังก์ชันตัวนี้สามารถตั้งชื่ออะไรก็ได้
พารามิเตอร์ตัวที่สาม	คือ ค่า 32 บิต ซึ่งต้องการส่งให้กับ callback function เช่น ส่งค่าพอยเตอร์ที่ชี้ไปยังตัวแปรของ GUID เพื่อที่จะกำหนดค่า keyboard instance identifier ในกับตัวแปรเหล่านี้
พารามิเตอร์ตัวที่สี่	คือ ค่าสถานะที่บอกว่าจะให้ระบุอุปกรณ์ทั้งหมดหรือเฉพาะที่เชื่อมต่ออยู่เท่านั้น (<code>DIEDFL_ALLDEVICES</code> หรือ <code>DIEDFL_ATTACHEDONLY</code>)

ถ้าต้องการที่จะใช้อุปกรณ์หลายชนิด callback function สามารถในการกำหนดค่าเริ่มต้นให้กับอุปกรณ์แต่ละชนิดได้ และจะได้รับค่า GUID ของอุปกรณ์จาก callback นอกจากนั้นยังสามารถประมวลผลอย่างอื่นได้อีกเช่น การค้นหาชนิดของอุปกรณ์บางชนิด หรือ การเพิ่มชื่อของอุปกรณ์ที่รู้จักเข้าในบัญชีรายชื่อ

ตัวอย่าง

```

BOOL hasEnhanced;

BOOL CALLBACK DIEDenumKbdCallback(LPCDIDEVICEINSTANCE lpddi, LPVOID pvRef)
{
    *(GUID*) pvRef = lpddi->guidInstance;

    if (GET_DIDEVICE_SUBTYPE(lpddi->dwDevType) == DIDEVTYPEKEYBOARD_PCENH)
    {
        hasEnhanced = TRUE;

        return DIENUM_STOP;
    }

    return DIENUM_CONTINUE;
} // End of callback

```

7.5 ไดรฟ์อินพุตไวด์ (DirectInput Device)

การติดตั้งไวด์ (Device Setup)

แอปพลิเคชันจะต้องรับค่า DirectInput device object สำหรับแต่ละอุปกรณ์ที่ต้องการอินพุตจากมัน นอกจากนั้น จะต้องเตรียมอุปกรณ์แต่ละตัวให้พร้อมที่จะใช้งานอย่างที่ต้องการ เช่น การกำหนดค่ารูปแบบข้อมูล รวมถึงการหาข้อมูลของอุปกรณ์แต่ละตัวและการเปลี่ยนแปลงคุณสมบัติของอุปกรณ์นั้นๆ

ขั้นตอนต่อไปนี้เป็นขั้นตอนในกระบวนการติดตั้ง ซึ่งจะต้องทำตามขั้นตอนเสมอ แต่สามารถเพื่อเติมขั้นตอนเข้าไปได้ถ้าต้องการข้อมูลจากอุปกรณ์เพิ่มเติม

1. สร้าง DirectInput device (จำเป็น)
2. รับค่าความสามารถของอุปกรณ์ (device capability) (ตัวเลือก)
3. ระบุเป็นอักษร, ปุ่ม หรือแกน ของอุปกรณ์ (ตัวเลือก)
4. กำหนดค่าระดับการทำงานร่วมกัน (cooperative level) (แนะนำว่าควรทำ)
5. กำหนดค่ารูปแบบของข้อมูล (จำเป็น)
6. กำหนดค่าคุณสมบัติของอุปกรณ์ (ถ้าใช้บัฟเฟอร์จะต้องกำหนดขนาดของมัน)
7. เมื่อใดที่พร้อมที่จะรับข้อมูล ให้รับอุปกรณ์มา (acquire device) (จำเป็น)

การสร้างอุปกรณ์ไดเรกอินพุต (Creating a DirectInput Device)

เพื่อที่จะรับข้อมูลจากอุปกรณ์ ขั้นแรกจะต้องสร้างออปเจกต์ (object) ที่แทนอุปกรณ์นั้นก่อน
 IDirectInput8::CreateDevice เป็นเมทอดที่ใช้ในการรับค่าพอยเตอร์ที่ชี้ไปยังอินเทอร์เฟซ (interface)
 IDirectInputDevice8 เมทอดของอินเทอร์เฟซนี้ยังใช้ในการควบคุมและรับข้อมูลจากอุปกรณ์อีกด้วย
 ตัวอย่าง

```
LPDIRECTINPUTDEVICE8 lpdiKeyboard;  
lpdi->CreateDevice(GUID_SysKeyboard, &lpdiKeyboard, NULL);
```

พารามิเตอร์ตัวแรก คือ อินสแตนซ์ (instance) ของ GUID ที่ใช้จำแนกอินสแตนซ์ของอุปกรณ์
 สำหรับอินเทอร์เฟซที่จะสร้าง ใน DirectInput มี GUID 2 ตัวที่ได้รับการ
 กำหนดค่าไว้ล่วงหน้าแล้ว คือ GUID_SysMouse และ GUID_SysKeyboard
 ซึ่งแทนเมาส์และคีย์บอร์ดของระบบ ค่า GUID 2 ตัวนี้สามารถส่งผ่านให้กับเม
 ทอด CreateDevice ได้เลย สำหรับอุปกรณ์ชนิดอื่นๆ นอกจากเมาส์และ
 คีย์บอร์ดแล้ว ค่าอินสแตนซ์ GUID จะได้มาจากเมทอด EnumDevices หรือ
 EnumDevicesBySemantics

ความสามารถของอุปกรณ์

การที่จะรับข้อมูลจากอุปกรณ์จำเป็นต้องรู้ความสามารถบางอย่างที่อุปกรณ์มี เช่น เมาส์เชื่อม
 ต่ออยู่กับเครื่องคอมพิวเตอร์อยู่หรือไม่ เป็นต้น ค่าเหล่านี้หาได้จากการเรียกเมทอด IDirectInputDevice8::
 GetCapabilities ซึ่งจะส่งข้อมูลโครงสร้าง DIDEVCAPS กลับคืนมา
 ตัวอย่าง

```
DIDEVCAPS DIMouseCaps;  
HRESULT hr;  
BOOLEAN WheelAvailable;  
  
DIMouseCaps.dwSize = sizeof(DIDEVCAPS);  
hr = lpdiMouse->GetCapabilities(&DIMouseCaps);  
WheelAvailable = ((DIMouseCaps.dwFlags & DIDC_ATTACHED)  
    && (DIMouseCaps.dwAxes > 2));
```

นอกจากนี้ยังมีอีกวิธีการหนึ่งในการตรวจสอบปุ่มหรือแกนโดยการเรียก
 IDirectInputDevice8::GetObjectInfo จากออปเจกต์นั้น ถ้าเมทอดส่งค่ากลับคืนเป็น
 DIERR_OBJECTNOTFOUND แสดงว่าไม่มีออปเจกต์นั้นอยู่
 ตัวอย่าง การตรวจสอบว่ามีแกน z อยู่หรือไม่

```
DIDeviceObjectInstance didoi;

didoi.dwSize = sizeof(DIDeviceObjectInstance);

hr = lpdiMouse->GetObjectInfo(&didoi, DIMOFS_Z, DIPH_BYOFFSET);

WheelAvailable = SUCCEEDED(hr);
```

7.6 ระดับการทำงานร่วมกัน (Cooperative Levels)

ระดับการทำงานร่วมกันของอุปกรณ์ใช้ในการกำหนดว่าจะมีการใช้งานอินพุตร่วมกันได้หรือไม่
 ระหว่างแอปพลิเคชันและกับระบบปฏิบัติการวินโดวส์ ระดับการทำงานร่วมกันสามารถกำหนดได้โดยการ
 เรียกเมทอด IDirectInputDevice8::SetCooperativeLevel

ตัวอย่าง

```
/* hwnd is the top-level window handle. */

lpdiDevice->SetCooperativeLevel(hwnd, DISCL_NONEXCLUSIVE | DISCL_FOREGROUND)
```

ค่าพารามิเตอร์ตัวแรก คือ ค่าแฮนเดิลของวินโดวส์ที่อยู่บนสุด (top-level window) ซึ่งจะได้รับการติดต่อ
 กับอุปกรณ์

ค่าพารามิเตอร์ตัวที่สอง คือ ค่าสถานะที่จะกำหนดให้กับระดับการใช้งานร่วมกัน

แม้ว่า DirectInput จะได้กำหนดค่าโดยปริยายให้กับระดับการใช้งานร่วมกันของอุปกรณ์ไว้แล้ว
 ก็จะต้องกำหนดค่านี้ใหม่เนื่องจากเมทอดนี้เป็นเมทอดเดี่ยวที่จะให้ค่าแฮนเดิลของวินโดวส์แก่ไคลเอนท์
 เนื่องจากถ้าไคลเอนท์ไม่มีค่าแฮนเดิลจะไม่สามารถตอบสนองต่อเหตุการณ์ที่มีวินโดวส์เมซเซจเข้ามา
 เกี่ยวข้องได้

ค่าแสดงสถานะ	หมายเหตุ
DISCL_NONEXCLUSIVE DISCL_BACKGROUND	ค่ากำหนดโดยปริยาย
DISCL_NONEXCLUSIVE DISCL_FOREGROUND	
DISCL_EXCLUSIVE DISCL_FOREGROUND	
DISCL_EXCLUSIVE DISCL_BACKGROUND	ไม่สามารถใช้กับคีย์บอร์ดและเมาส์ได้

ตารางที่ 7-1 แสดงค่าแสดงสถานะที่เป็นไปได้

ในกรณีของคีย์บอร์ด สามารถเพิ่ม DISCL_NOWINKEY ร่วมกับ DISCL_NONEXCLUSIVE ได้ โดยค่าแสดงสถานะนี้จะทำให้เป็นคีย์โลโก้วินโดว์ใช้การไม่ได้ ใน exclusive mode เป็นคีย์โลโก้วินโดว์จะใช้การไม่ได้เสมอ

ค่าแสดงสถานะของระดับการใช้งานร่วมกัน

Foreground และ Background

Foreground คือ อุปกรณ์อินพุตจะใช้งานได้ก็ต่อเมื่อแอปพลิเคชันทำงานอยู่ในเบื้องหน้า หรือเรียกอีกอย่างหนึ่งว่าได้รับโฟกัส ถ้าแอปพลิเคชันไปทำงานเบื้องหลังเมื่อใดอุปกรณ์อินพุตจะไม่สามารถใช้งานจากแอปพลิเคชันได้

Background คือ แอปพลิเคชันสามารถใช้อุปกรณ์อินพุตได้ตลอดเวลาไม่ว่าจะอยู่เบื้องหน้าหรือเบื้องหลัง

Exclusive และ Nonexclusive

ในความเป็นจริงแล้วแม้ว่าแอปพลิเคชันจะใช้งานอุปกรณ์แบบแต่เพียงผู้เดียวไม่ได้หมายความว่าไม่มีแอปพลิเคชันอื่นที่สามารถรับอินพุตได้ นั่นคือ ไม่มีแอปพลิเคชันใดที่สามารถใช้อุปกรณ์อินพุตแต่เพียงผู้เดียวได้โดยแท้จริง ตัวอย่างเช่น การกดแป้นพิมพ์ ALT+TAB

7.7 การระบุอุปกรณ์ (Device Object Enumeration)

เป็นสิ่งจำเป็นเนื่องแอปพลิเคชันจำเป็นต้องรู้ว่ามียูนิตหรือคันทริกเกอร์ใดที่มีอยู่บนอุปกรณ์ การระบุอุปกรณ์มีวิธีการคล้ายคลึงกับการระบุอุปกรณ์ โดยสามารถใช้เมทอด `IdirectInputDevice8::EnumObjects` และ `IdirectInputDevice8::GetCapabilities` แทน

ตัวอย่าง

```
char szName[MAX_PATH];

BOOL CALLBACK DIEnumDeviceObjectsCallback(
    LPCDIDEVICEOBJECTINSTANCE lpddoi, LPVOID pvRef)
{
    lstrcpy(szName, lpddoi->tszName);
    // Now add szName to a list or array.
    return DIENUM_CONTINUE;
}
```

7.8 รูปแบบข้อมูลของอุปกรณ์ (Device Data Formats)

ถ้าไม่มีการใช้ Action Mapping การกำหนดรูปแบบข้อมูลสำหรับอุปกรณ์เป็นขั้นตอนที่สำคัญในการได้มาและเริ่มต้นการใช้งานอุปกรณ์ ในไดเรกอินพุตมีเมทอดจำนวนมากที่ใช้เกี่ยวข้องกับการกำหนดรูปแบบข้อมูล

เมทอด IDirectInputDevice8::SetDataFormat กำหนดว่าออปเจ็กต์อุปกรณ์ใดที่จะใช้และวิธีการที่ข้อมูลจะเรียงลำดับ โครงสร้างข้อมูล DIDATAFORMAT และ DIOBJECTDATAFORMAT แสดงวิธีการกำหนดค่าสำหรับอุปกรณ์ที่ไม่มีมาตรฐาน ส่วนอุปกรณ์ที่เป็นมาตรฐาน เช่น จอยสติค, เมาส์ และคีย์บอร์ด ได้มีการกำหนดค่าไว้แล้วดังนี้ c_dfDIJoystick, c_dfDIJoystick2, c_dfDIKeyboard, c_dfDIMouse และ c_dfDIMouse2 ซึ่งสามารถส่งให้กับเมทอด SetDataFormat ได้ทันที

ตัวอย่าง

```
lpdiMouse->SetDataFormat(&c_dfDIMouse);
```

7.9 ข้อมูลจากอุปกรณ์ไดเรกอินพุต

ข้อมูลบัฟเฟอร์และอิมมีเดียท (Buffered and Immediate data)

ในไดเรกอินพุตข้อมูลมี 2 แบบ คือ บัฟเฟอร์ และ อิมมีเดียท โดยบัฟเฟอร์จะบันทึกเหตุการณ์ที่เกิดขึ้นเก็บไว้จนกว่าแอปพลิเคชันจะมารับค่าเหล่านั้นไป (อาจเกิดบัฟเฟอร์โอเวอร์โฟลว์ (Buffer Overflow) ถ้ากำหนดขนาดบัฟเฟอร์เล็กเกินไปทำให้สูญเสียข้อมูลใหม่ๆ ไป) และอิมมีเดียทที่จะรับเฉพาะข้อมูลสถานะปัจจุบันเท่านั้น

7.10 การสำรวจข้อมูล (Polling)

การสำรวจข้อมูลจากอุปกรณ์อินพุตโดยปกติจะต้องทำให้มีเซเชนจ์ลูป (message loop) เนื่องจากจอยสติคและอุปกรณ์เกมส์อื่นๆ ไม่ได้สร้างฮาร์ดแวร์อินเทอร์รัพ (Hardware Interrupt) หรือส่งค่าใดๆ กลับคืนมาให้ จากกว่าจะมีการเรียกเมทอด `IdirectInputDevice8::Poll` (`Poll` ในที่นี้ไม่ได้หมายความว่าไปรับข้อมูลมาแต่เป็นการทำให้มีข้อมูลไว้ใช้งาน)